University of Asia Pacific

Department of Computer Science and Engineering

CSE 402:Operating System Lab

LAB REPORT

Submitted by: Shad Bin Moshiur

Student ID: 23101143

Section: c-2

Lab Task: 03

Submitted to:
Atia Rahman Orthi
Lecturer,
Department of Computer Science and Engineering

Date of Submission: 09/08/2026

Operating Systems Lab Report

CPU Scheduling: FCFS vs SJF (Non-Preemptive)

1. Objective

To implement First Come First Serve (FCFS) and Shortest Job First (SJF, non-preemptive) CPU scheduling algorithms in Python, compute Completion Time (CT), Turnaround Time (TAT) and Waiting Time (WT) for a given set of processes, and compare the performance of both algorithms.

2. Theory

2.1 First Come First Serve (FCFS)

FCFS is the simplest CPU scheduling algorithm. Processes are executed strictly in the order they arrive in the ready queue (i.e., sorted by Arrival Time). The CPU is allocated to a process on a non-preemptive basis, meaning once a process starts execution it runs to completion before the next process begins.

2.2 Shortest Job First (SJF) — Non-Preemptive

SJF selects, from the set of processes that have already arrived, the one with the smallest Burst Time (BT) to execute next. It is non-preemptive here, so once a process starts, it is not interrupted. SJF is provably optimal for minimizing average waiting time among non-preemptive algorithms, since running the shortest available job first reduces the time other jobs spend waiting behind long jobs.

3. Input Data (Process Set)

PID	Arrival Time (AT)	Burst Time (BT)
P1	3	3
P2	2	5
P3	5	4
P4	1	3
P5	6	2

4. Python Implementation

4.1 FCFS Function

```
def fcfs(processes):
    procs = sorted(processes, key=lambda x: x[1])  # sort by AT
    time = 0
    result = []
    for pid, at, bt in procs:
        start = max(time, at)
        ct = start + bt
        tat = ct - at
        wt = tat - bt
        result.append([pid, at, bt, ct, tat, wt])
        time = ct
    return result
```

4.2 SJF (Non-Preemptive) Function

```
def sjf_non_preemptive(processes):
    n = len(processes)
    is_done = [False] * n
    current_time = 0
    completed = 0
    result = []

    while completed != n:
        idx = -1
        min_bt = float('inf')
        for i in range(n):
            pid, at, bt = processes[i]
            if at <= current_time and not is_done[i] and bt < min_bt:
                min_bt = bt
                idx = i
        if idx == -1:
            current_time += 1
            continue
        pid, at, bt = processes[idx]
        current_time += bt
        ct = current_time
        tat = ct - at
        wt = tat - bt
        result.append([pid, at, bt, ct, tat, wt])
        is_done[idx] = True
        completed += 1
    return result
```

4.3 Driver Code

```
processes = [
    ["P1", 3, 3],
    ["P2", 2, 5],
    ["P3", 5, 4],
    ["P4", 1, 3],
    ["P5", 6, 2],
]

fcfs_result = fcfs(processes)
sjf_result = sjf_non_preemptive(processes)

fcfs_tat, fcfs_wt = print_table("FCFS Result", fcfs_result)
sjf_tat, sjf_wt = print_table("SJF Result", sjf_result)

print("\n--- Comparison ---")
print(f"{'Metric':<20}{'FCFS':<10}{'SJF':<10}")
print(f"{'Avg TAT':<20}{fcfs_tat:<10.2f}{sjf_tat:<10.2f}")
print(f"{'Avg WT':<20}{fcfs_wt:<10.2f}{sjf_wt:<10.2f}")
```

5. Output

5.1 FCFS Result

PID	AT	BT	CT	TAT	WT
P1	3	3	12	9	6
P2	2	5	9	7	2
P3	5	4	16	11	7
P4	1	3	4	3	0
P5	6	2	18	12	10

Average TAT = 8.40 Average WT = 5.00

5.2 SJF Result

PID	AT	BT	CT	TAT	WT
P1	3	3	7	4	1
P2	2	5	18	16	11
P3	5	4	13	8	4
P4	1	3	4	3	0
P5	6	2	9	3	1

Average TAT = 6.80 Average WT = 3.40

5.3 Comparison

Metric	FCFS	SJF
Average TAT	8.40	6.80
Average WT	5.00	3.40

6. Discussion

6.1 Why is FCFS Required in CPU Scheduling?

- Simplicity: FCFS is the easiest scheduling algorithm to understand and implement — it uses a simple FIFO (First-In-First-Out) queue, so it is the natural starting point for teaching CPU scheduling concepts.
- Fairness in arrival order: Every process is guaranteed to be served strictly in the order it requests the CPU, so no process is ever starved or repeatedly skipped in favor of newer arrivals.
- No starvation: Because ordering never changes once a process is in the queue, every process is guaranteed to eventually get the CPU — a property that priority-based or shortest-job-based algorithms do not always guarantee.
- Baseline for comparison: FCFS provides a simple reference algorithm against which more advanced algorithms (SJF, Round Robin, Priority Scheduling) can be evaluated in terms of average waiting time and turnaround time.
- Low overhead: FCFS requires no estimation of future burst times and no reordering of the ready queue, so it has minimal scheduling overhead — useful in situations where predictability of process order matters more than optimizing average wait.

6.2 Why Does SJF Perform Better Than FCFS?

- Minimizes average waiting time: SJF is mathematically proven to give the minimum possible average waiting time among non-preemptive scheduling algorithms, because it always picks the job that will free up the CPU soonest.
- Shorter jobs are not stuck behind longer ones: In FCFS, a short process arriving right after a long process must wait for the entire long process to finish. SJF avoids this by letting short jobs "jump ahead" of longer jobs that arrived earlier, as seen with P4 (BT = 3) and P5 (BT = 2) finishing very early in the SJF result.
- Reduces the convoy effect: In FCFS, a long process at the front of the queue forces every other process behind it to wait, similar to a slow vehicle blocking traffic on a single-lane road (the convoy effect). SJF reduces this bottleneck by prioritizing short jobs.
- Result confirms the theory: In this experiment, SJF achieved an average TAT of 6.80 versus 8.40 for FCFS, and an average WT of 3.40 versus 5.00 for FCFS — both metrics are noticeably lower for SJF, confirming its better performance for this process set.
- Trade-off: SJF's better average performance comes at the cost of needing to know (or estimate) burst times in advance, and it can theoretically starve longer processes if short processes keep arriving — a limitation FCFS does not have.

7. Conclusion

Both FCFS and SJF were implemented and tested on the same set of five processes. The results show that SJF (non-preemptive) outperforms FCFS in this scenario, reducing the average turnaround time from 8.40 to 6.80 and the average waiting time from 5.00 to 3.40. This confirms the theoretical expectation that SJF minimizes average waiting time by prioritizing shorter jobs, while FCFS, despite being simpler and fairer in arrival order, suffers from the convoy effect when short processes are queued behind long ones.

GitHub Link:

https://github.com/Moshiur1143/Operating-System-Repository-23101143-/tree/main/Lab-03-SJF-Comparison-with-FCFS/Report